SELF-HOSTING GUIDE · V1.0

SecureVault Self-Hosting

Deploy. Own. Earn.
A complete guide to running SecureVault on your own infrastructure.

React 19 | Express 4 | tRPC 11 | MySQL | Amazon S3 | AES-256

SecureVault v1.0 · MIT License · Full Source Included

AES-256 Encrypted
Client-side · Zero knowledge

OVERVIEW

A password manager you own completely — no subscriptions, no middlemen.

- **One-time $4.99 purchase** — users own it for life, no recurring fees
- **AES-256 client-side encryption** — data is encrypted before it ever leaves the device
- **Stores everything** — passwords, secure notes, credit cards, identity documents, and private images
- **Built-in referral program** — earn $1 for every friend who buys through your personal link
- **True data ownership** — when self-hosted, your users' data never touches anyone else's infrastructure

PILLAR 01

Your Code

MIT-licensed · modify freely

PILLAR 02

Your Data

AES-256 · your DB + your S3

PILLAR 03

Your Revenue

$4.99/user · $1/referral · direct to Stripe

PREREQUISITES

A lean, modern stack that runs on a $4/month server.

REQUIREMENT	MINIMUM	RECOMMENDED
Node.js	v18+	v22 LTS
Package Manager	pnpm v8+	pnpm v10+
Database	MySQL 8.0+	PlanetScale / TiDB Cloud
RAM	512 MB	1 GB+
Storage	1 GB	5 GB+
Operating System	Ubuntu 20.04+	Ubuntu 22.04 LTS

"No Docker required to get started — a plain Ubuntu VPS is all you need. Docker support is also included for advanced deployments."

CONFIGURATION

Environment Variables: Your Control Panel

Every configuration decision flows through a single `.env` file. Get this right first — if it is wrong, nothing works.

GROUP 01

DATABASE_URL

MySQL connection string — host, user, password, and database name in a single URL.

GROUP 02

JWT_SECRET

Signs all session cookies. Generate once, never change it.

```
openssl rand -hex 64
```

GROUP 03

STRIPE_SECRET_KEY & STRIPE_PUBLISHABLE_KEY

Your Stripe API keys for processing $4.99 payments and $1 referral rewards.

GROUP 04

AWS_ACCESS_KEY_ID & S3_BUCKET

Amazon S3 credentials powering the Image Vault — access key, secret, region, and bucket name.

GROUP 05

NODE_ENV & App Config

Set `NODE_ENV=production` plus app title, logo URL, and OAuth credentials.

GROUP 06

STRIPE_WEBHOOK_SECRET

Signing secret from Stripe dashboard — verifies webhook authenticity. Starts with `whsec_`

⚠ Never commit `.env` to Git. Add it to `.gitignore` immediately. A template file `SecureVault_Environment_Variables.txt` is included in your download package.

STEP 01 · DATABASE

Choose your database path — from zero-config cloud to full control.

OPTION A

PlanetScale RECOMMENDED

Free tier · Zero management · Live in 2 minutes

- Serverless MySQL — no server to manage
- Generous free tier for small apps
- Automatic branching and schema migrations

```
Create account → New DB
Connect → Node.js
Copy URL → paste into .env
```

OPTION B

TiDB Cloud

Serverless · MySQL-compatible · Asia-Pacific friendly

- Free serverless tier available
- Fully MySQL-compatible — no code changes
- Best latency for Asia-Pacific regions

```
Create cluster → Connect panel
Select Node.js driver
Copy URL → paste into .env
```

OPTION C

Self-Hosted MySQL

Full control · Best for VPS deployments

- Complete data sovereignty on your server
- No external dependencies or vendor lock-in
- Requires managing backups and security patches

```
sudo apt install mysql-server
Create DB + user
Set DATABASE_URL in .env
```

FINAL STEP — ALL OPTIONS

```
pnpm db:push
```

Drizzle ORM generates and applies all migrations automatically — schema is live in seconds.

STEP 06 · STORAGE

Amazon S3: Powering the Image Vault

01 Create S3 Bucket

Name it `securevault-images`, choose your region, uncheck "Block all public access"

02 Create IAM User

Name: `securevault-s3` · Attach policy: `AmazonS3FullAccess`

03 Generate Access Keys

Copy Access Key ID + Secret Access Key → paste into `.env` as `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`

04 Set CORS Policy

Allow GET, PUT, POST, DELETE from your domain — see code example →

CORS POLICY · S3 BUCKET

```
[
  {
    "AllowedOrigins": [
      "https://yourdomain.com"
    ],
    "AllowedMethods": [
      "GET", "PUT",
      "POST", "DELETE"
    ],
    "AllowedHeaders": ["*"],
    "MaxAgeSeconds": 3000
  }
]
```

COST REFERENCE

"S3 storage costs pennies per GB — a typical user's image vault costs less than $0.01/month. The free tier covers 5 GB for the first 12 months."

PAYMENTS

Stripe Setup: Getting Paid

1. **Create account** at stripe.com and complete KYC verification to enable payouts
2. Go to **Developers** → **API Keys** → copy Publishable Key + Secret Key → paste into `.env`
3. Product price is defined in `server/products.ts` — currently **499 cents ($4.99)**. Edit this file to change the price.
4. Use **test keys** (`sk_test_...`) during development. Swap for live keys before launch.
5. Register your **Stripe webhook** after deploying — covered in detail on Slide 13.

TEST CARD DETAILS

Card Number	4242 4242 4242 4242
Expiry	Any future date
CVV	Any 3 digits

APP PRICE

$4.99 one-time · lifetime access

Defined in server/products.ts → 499 cents

$1 Per Referral

Every successful referral earns the referrer $1 — tracked automatically via the Stripe `checkout.session.completed` webhook event.

AUTHENTICATION

Two paths to user authentication — simple or fully custom.

OPTION A — RECOMMENDED

Keep Manus OAuth

- ✓ **Zero code changes** required
- ✓ Copy **5 OAuth env vars** from your Manus project settings
- ✓ Existing credentials work on your self-hosted instance

```
VITE_APP_ID=...
OAUTH_SERVER_URL=...
VITE_OAUTH_PORTAL_URL=...
OWNER_OPEN_ID=...
OWNER_NAME=...
```

OPTION B — ADVANCED

Replace with External Provider

- Modify **server/_core/oauth.ts**
- Modify **client/src/_core/hooks/useAuth.ts**
- Clerk is the easiest integration — free tier supports **10,000 users/month**

PROVIDER	FREE TIER	COMPLEXITY	BEST FOR
Google OAuth 2.0	Unlimited	Medium	Apps where users already have Google accounts
Auth0	7,500 users/month	Medium	Enterprise-grade auth with advanced rules
Clerk	10,000 users/month	Low	Fastest integration with pre-built UI components

DEPLOYMENT OPTION A

Deploy on Railway: Zero to Live in 10 Minutes

1 **Create a Railway account**
Sign up at railway.app — free tier available, no credit card required to start

2 **New Project → Deploy from GitHub**
Connect your GitHub account and select the SecureVault repository

3 **Add MySQL plugin**
Click + New → Database → MySQL. Railway provisions and auto-links it to your app

4 **Paste environment variables**
Go to the Variables tab → paste all values from your `.env` file

5 **Build & start commands auto-detected**
Railway detects Node.js and sets `pnpm build` and `node dist/index.js` automatically

6 **Click Deploy**
Railway builds and deploys your app — live at a `*.railway.app` URL in 2–3 minutes

7 **Add your custom domain**
Settings → Domains → enter your domain. SSL certificate provisioned automatically

- ✓ SSL automatic — no Certbot needed
- ✓ No Nginx configuration
- ✓ No PM2 process manager
- ✓ No SSH required

DEPLOYMENT · OPTION 2

Deploy on VPS: Full Control

1	2	3	4	5
Prepare Server	Upload & Build	Start with PM2	Configure Nginx	Enable HTTPS

01 Prepare Server — install Node.js 22, pnpm, PM2, Nginx

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo bash -
sudo apt install -y nodejs nginx
npm install -g pnpm pm2
```

02 Upload & Build — clone repo, create .env, install, migrate, compile

```
git clone https://github.com/you/securevault.git
pnpm install && pnpm db:push && pnpm build
```

03 Start with PM2 — keeps app alive across reboots

```
pm2 start dist/index.js --name securevault
pm2 save && pm2 startup
```

04 & 05 Nginx reverse proxy + free SSL via Certbot / Let's Encrypt

```
# Nginx: proxy port 80 → localhost:3000
sudo certbot --nginx -d yourdomain.com
# Auto-renews before expiry
```

VPS PROVIDER COSTS

Hetzner CX11	1 GB RAM · 20 GB SSD	$4/mo
Linode Nanode	1 GB RAM · 25 GB SSD	$5/mo
DigitalOcean	1 GB RAM · 25 GB SSD	$6/mo
Vultr Cloud	1 GB RAM · 25 GB SSD	$6/mo

PM2 LOG COMMANDS

```
pm2 status # view all processes
pm2 logs securevault # tail live logs
pm2 restart securevault # restart app
```

> *"A $4/month Hetzner server comfortably handles hundreds of concurrent SecureVault users — no scaling needed at launch."*

DEPLOYMENT · OPTION 3

Deploy with Docker: Build Once, Run Anywhere

TWO COMMANDS TO DEPLOY

```
# Step 1 — Build the image
docker build -t securevault .

# Step 2 — Run the container
docker run -d \
  --name securevault \
  -p 3000:3000 \
  --env-file .env \
  --restart unless-stopped \
  securevault
```

A **Docker Compose** file is included for running the app alongside Nginx in a single configuration — no manual networking required.

```
docker-compose up -d
```

COMPATIBLE PLATFORMS

Kubernetes
Deploy as a pod with your existing cluster

AWS ECS
Push to ECR, run as a Fargate task

Google Cloud Run
Serverless container — scales to zero when idle

DOCKERFILE HIGHLIGHTS

- **Multi-stage build** — compile stage + lean production image, no dev dependencies in final image
- **Health check included** — pings `/api/trpc/auth.me` every 30 seconds
- **Auto-restart** on failure with `--restart unless-stopped`

STEP 12 · DOMAIN

Custom Domain & SSL — Your brand, free HTTPS.

DNS CONFIGURATION

TYPE	NAME	VALUE
A	@ (root)	your.server.ip.address
A	www	your.server.ip.address
CNAME	vault	yourapp.railway.app

SSL OPTIONS

VPS

`Certbot + Let's Encrypt`

Free, auto-renews. One command: `certbot --nginx`

Railway

`Automatic`

Provisioned instantly when custom domain is added in dashboard

Render

`Automatic`

Provisioned instantly when custom domain is added in dashboard

Free SSL Command (VPS):

`sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com`

Auto-renews before expiry via cron. No manual intervention needed.

EMBEDDING ON YOUR EXISTING SITE

Point `vault.yourwebsite.com` to your SecureVault instance via a CNAME record. Users see it as part of your brand — no iframes, no redirects, just your domain.

CERTBOT INSTALL (UBUNTU)

```
sudo apt install certbot
sudo apt install python3-certbot-nginx
sudo certbot --nginx \
  -d yourdomain.com
```

STEP 13 · CRITICAL

Stripe Webhook: The Step Most People Forget

USER PAYS
Stripe processes the $4.99 payment successfully

STRIPE FIRES EVENT
checkout.session.completed sent to your webhook URL

WITHOUT WEBHOOK
Your app never receives it — user stays locked out

WITH WEBHOOK
User marked as paid in DB — vault unlocks instantly

4-STEP SETUP

1. Go to **Stripe Dashboard** → **Developers** → **Webhooks** → **Add Endpoint**
2. Enter your webhook URL: `https://yourdomain.com/api/stripe/webhook`
3. Select events: `checkout.session.completed` + `payment_intent.succeeded`
4. Copy the **Signing Secret** (`whsec_...`) → add as `STRIPE_WEBHOOK_SECRET` in .env → redeploy

WEBHOOK URL FORMAT

```
POST https://yourdomain.com/api/stripe/webhook

Events to select:
  ✓ checkout.session.completed
  ✓ payment_intent.succeeded

Signing Secret:
  STRIPE_WEBHOOK_SECRET=whsec_...
```

#1 cause of "I paid but can't access the app" support tickets. Do not skip this step — register the webhook before announcing your launch.

FINAL STEP · PRE-LAUNCH

Going Live Checklist

- ☑ **NODE_ENV set to production**
  Confirm `NODE_ENV=production` in your .env — enables security headers and disables debug output

- ☑ **pnpm db:push completed**
  All schema migrations applied to the production database — run `pnpm db:push` after every deploy

- ☑ **HTTPS active on all routes**
  Padlock visible in browser — HTTP requests redirect to HTTPS automatically

- ☑ **Stripe webhook registered**
  Endpoint `/api/stripe/webhook` added in Stripe Dashboard with `STRIPE_WEBHOOK_SECRET` in .env

- ☑ **JWT_SECRET is strong and unique**
  Minimum 32 random characters — generate with `openssl rand -hex 32`

- ☑ **DATABASE_URL points to production DB**
  Not localhost — must be your live MySQL/TiDB connection string with SSL enabled

- ☑ **Custom domain live**
  DNS A/CNAME records propagated — verify with `nslookup yourdomain.com`

- ☑ **Stripe live keys configured**
  Replace test keys (`sk_test_`) with live keys (`sk_live_`) after completing Stripe KYC

- ☑ **S3 storage configured**
  `AWS_ACCESS_KEY_ID` , `AWS_SECRET_ACCESS_KEY` , and `S3_BUCKET_NAME` set and bucket accessible

- ☑ **End-to-end purchase tested**
  Complete a full flow: sign up → pay with `4242 4242 4242 4242` → vault unlocks → referral link works

All 10 checks complete — you are ready to launch.
Share your referral link and start earning $1 per friend who purchases.

10	$4.99	$1
CHECKS	PER SALE	PER REFERRAL

REFERENCE · STEP 14

Troubleshooting Guide

ERROR App won't start

Cause: Missing or malformed environment variable in .env

```
cat .env | grep -v '^#' | grep -v '^$' # verify all keys set
```

ERROR DB connection error

Cause: Wrong DATABASE_URL format or DB server unreachable

```
mysql -u user -p -h host dbname # test connection directly
```

ERROR Stripe webhook fails

Cause: STRIPE_WEBHOOK_SECRET mismatch or wrong endpoint URL

```
Stripe Dashboard → Webhooks → Reveal Signing Secret → update .env
```

ERROR Images not uploading

Cause: AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY / S3_BUCKET incorrect

```
Verify all AWS_* and S3_BUCKET env vars match your bucket region
```

ERROR Login not working

Cause: VITE_APP_ID or OAUTH_SERVER_URL misconfigured

```
Confirm VITE_APP_ID matches your Manus OAuth app registration
```

ERROR pnpm db:push fails

Cause: DB not reachable from server or firewall blocking port 3306

```
sudo ufw allow 3306 # or check cloud provider firewall rules
```

ERROR 502 Bad Gateway

Cause: App process crashed or not running on expected port

```
pm2 restart securevault && pm2 logs securevault --lines 20
```

LOG COMMANDS REFERENCE

```
pm2 logs securevault    # live app logs (VPS)
pm2 status              # process health overview
docker logs securevault # container logs (Docker)
sudo tail -f /var/log/nginx/error.log  # Nginx errors
```

CLOSING · OWNERSHIP SUMMARY

You Now Own Your Infrastructure

Your code. Your data. Your revenue. No middlemen.

01

Your Code

- **MIT-licensed** — use, modify, and distribute freely
- **Full source code** included — no black boxes, no vendor lock-in
- Export as ZIP anytime from the Management UI
- Deploy to **any server** — Railway, VPS, Docker, AWS, GCP

02

Your Data

- **AES-256 encryption** — all vault entries encrypted client-side
- **Zero-knowledge** — server never sees plaintext passwords
- Your own **MySQL database** — full ownership and control
- Images stored in **your own S3 bucket** — private by default

03

Your Revenue

- `$4.99` one-time lifetime purchase — low friction, high conversion
- `$1` referral reward per successful invite — viral growth built in
- **100% paid directly to you** via Stripe — no platform cut
- 30-day money-back guarantee (voided if referral reward claimed)

1 **Deploy Today**
Railway · VPS · Docker

2 **Share Your Link**
Twitter · WhatsApp · Email

3 **Start Earning**
$4.99 per user · $1 per referral

RESOURCES

Self-Hosting Guide (PDF)
Environment Variables Reference
Full Source Code (ZIP)